Structuri de Date
Laboratorul 6: Arbori Binari de Căutare

Dan Novischi, Mihai Nan

28 martie 2026

1. Introducere

Scopul acestui laborator îl reprezintă implementarea unui arbore binar de căutare. Obiectivul este acela de a vă familiariza cu lucrul cu arbori binari și de a face o comparație cu structurile ordonate definite în laboratoarele anterioare (mulțimi și liste ordonate).

Laboratorul are 3 cerințe care urmăresc:

- înțelegerea interfeței de lucru cu arborii binari de căutare (BST);
- implementarea funcțiilor aferente interfeței de lucru cu BST;
- implementarea unei funcții care determină cel mai apropiat strămoș comun pentru două noduri având cheile `value1` și `value2`.

2. Arbori binari de căutare

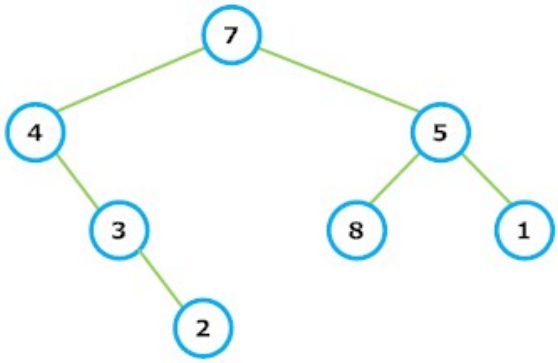


Figura 1: Arbore Binar Simplu

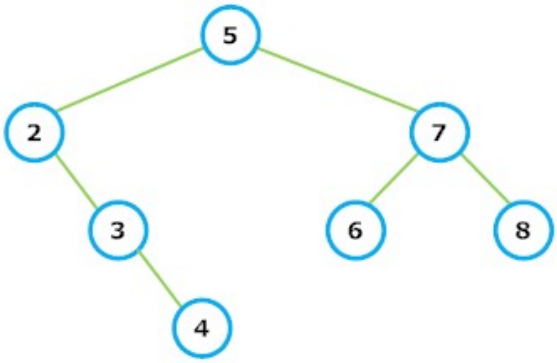


Figura 2: Arbore Binar de Căutare

Arborii sunt structuri de date ierarhice ale căror elemente se numesc **noduri** (similar cu nodurile listelor definite în laboratoarele anterioare). Nodurile stochează valorile datelor de interes (într-un câmp denumit **elem**) și conțin legături aferente pentru organizarea acestora sub formă ierarhică. Astfel, un arbore binar (vezi Figura 1) este o structură de date dinamică

ale cărui noduri pot avea cel mult două legături ierarhice.

Vocabular:

- Cel mai de sus nod din ierarhie se numește *rădăcina* arborelui. În Figura 1 rădăcina este dată de nodul cu $elem = 7$, respectiv în Figura 2 de nodul cu $elem = 5$.
- Dacă un nod x este ierarhic imediat sub alt nod y, atunci x se numește copil al nodului y, iar y se numește părinte al nodului x. În Figura 2 nodul cu $elem = 6$ este copil al nodului cu $elem = 7$. La rândul lui, nodul cu $elem = 7$ este părinte pentru nodurile cu $elem = 6$ și $elem = 8$.
- Nodurile care nu au copii se numesc frunze. În Figura 2, frunzele sunt nodurile cu elementele 4, 6 și 8.

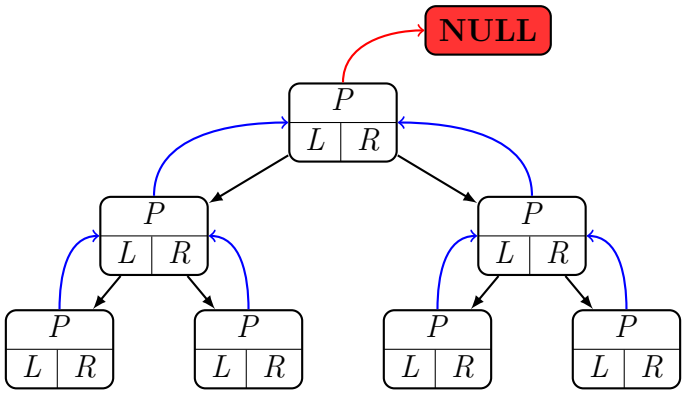
Un **arbore binar de căutare** este un arbore binar ale cărui noduri respectă următoarele relații de ordine:

```
node->left->elem < node->elem; // dacă node->left != NULL
node->right->elem > node->elem; // dacă node->right != NULL
```

Spre exemplu, arborele din Figura 2 este un arbore binar de căutare, deoarece toate elementele acestuia respectă relațiile de ordine de mai sus. Astfel, avem următoarele definiții pentru un abore și nodurile acestuia:

```
typedef struct node {
    Item value;
    struct node *left;
    struct node *right;
    struct node *parent;
} TreeNode, *Tree;
```

Reprezentarea grafică a variantei în care pentru fiecare nod păstrăm și un pointer la părintele nodului este următoarea:



Cerința 1 (1p) Analizați antetele funcțiilor definite în fișierul `tree.h` și parcurgeți descrierile lor.

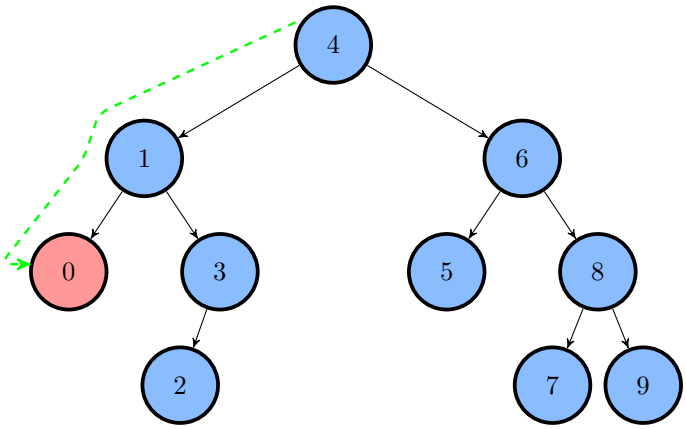
- `createTree`: Primește ca parametru un pointer la un nod ce reprezintă părintele și o valoare și returnează un pointer la un nod frunză ce va conține valoarea primită ca argument și va avea o legătură către părintele primit ca parametru.
- `init`: Similar cu `createTree`, dar este de tip `void`, iar rezultatul o să fie furnizat pe baza primului parametru de tip pointer la `Tree`.
- `isEmpty`: Verifică dacă un arbore este gol sau nu (1 – dacă este arbore vid sau 0 – dacă nu este arbore vid).
- `insert`: Primește un arbore și un `elem`. Funcția inserează valoarea în arbore, dacă ea nu există deja, ținând cont de proprietățile unui arbore binar de căutare. Pentru fiecare nod va trebui să realizați corect legătura cu părintele.
- `contains`: Primește un arbore și un `elem`. Funcția verifică dacă valoarea există în arbore (1 – dacă valoarea există în arbore sau 0 – dacă valoarea nu există în arbore).
- `minimum`: Funcție care determină nodul ce conține elementul minim dintr-un arbore binar de căutare. Funcția va întoarce `NULL` pentru arborele vid.
- `maximum`: Funcție care determină nodul ce conține elementul maxim dintr-un arbore binar de căutare. Funcția va întoarce `NULL` pentru arborele vid.
- `delete`: Funcție care primește ca argumente rădăcina unui arbore și o valoare și șterge nodul care conține respectiva valoare din arbore. Funcția va întoarce rădăcina arborelui rezultat după ștergere.
- `successor`: Funcție ce primește ca parametru rădăcina unui arbore și o valoare existentă în arbore și întoarce succesorul nodului ce conține valoarea indicată.
- `predecessor`: Funcție ce primește ca parametru rădăcina unui arbore și o valoare existentă în arbore și întoarce predecesorul nodului ce conține valoarea indicată.
- `destroyTree`: Funcție care dealocă întreaga memorie alocată pentru un arbore binar. Pointerul la rădăcina arborelui primit ca parametru va pointa către `NULL` după ce se va apela funcția

Cerința 2 (7p) Implementați funcțiile de mai sus în urmatoarea ordine: `init` pe baza funcției `createTree`, `insert`, `isEmpty`, `contains`, `minimum` și `maximum`, `successor`, `predecessor`, `delete`, `destroyTree`.

Indicații:

Determinarea elementului minim

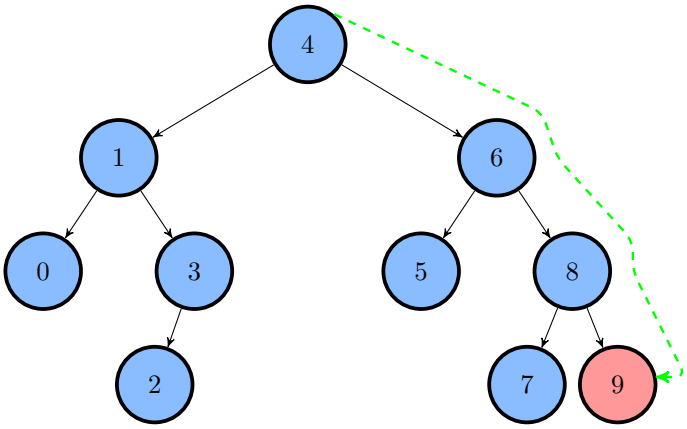
Pentru determinarea valorii minime, ne deplasăm în fiul stâng al rădăcinii. De acolo, în fiul stâng al acelui fiu ș.a.m.d., până când ajungem la un nod care nu mai are fiu stâng. Acest nod conține valoarea minimă din arbore. (Vezi fig. de mai jos)



Algorithm 1 FindMinimum

```
procedure FindMinimum(root)
    if root ≠ NULL then
        while root → left ≠ NULL do
            root ← root → left
        return root → value
```

Determinarea elementului maxim



Pentru determinarea nodului cu valoarea maximă procedăm similar, ca în cazul determinării minimului, avansând mereu spre fiul drept, până când găsim un nod care nu mai are niciun fiu drept. Acest nod va conține valoarea maximă. (Vezi fig. de mai sus)

Algorithm 2 FindMaximum

```
procedure FindMaximum(root)
    if root ≠ NULL then
        while root → right ≠ NULL do
            root ← root → right
        return root → value
```

Ștergerea unui nod dintr-un arbore binar de căutare

Operația de ștergere a unui nod presupune următoarele etape: se caută nodul care va fi șters, iar după ce am găsit nodul apar trei cazuri pe care le analizăm separat. Cele trei cazuri posibile sunt următoarele:

I. nodul care urmează să fie șters este o frunză (nu are fii);

II. nodul are un singur fiu;

III. nodul are doi fii.

Algorithm 3 Delete

```
procedure DELETE(root, x)
    if root = NULL then
        print(Nodul nu a fost gasit)
        return root
    if root → value > value then
        root → left ← delete(root → left, x)
    else if root → value < value then
        root → right ← delete(root → right, x)
    else                                                    ▷ Am găsit nodul căutat
        if root → left ≠ NULL and root → right ≠ NULL then
                                                            ▷ Nodul are 2 fii - cazul III
            tmp ← findMinimum(root → right)
            root → value ← tmp → value
            root → right ← delete(root → right, tmp → value)
        else                                ▷ Nodul are un fiu sau este frunză - cazurile I și II
            tmp ← root
            if root → left ≠ NULL then
                root ← root → left
            else
                root ← root → right
            free(tmp)
    return root
```

Succesorul în inordine

Succesorul unui nod, la traversarea în inordine, se defineşte ca:

1. elementul minim din sub-arborele drept al nodului, dacă nodul are sub-arbore drept.

2. dacă nodul nu are sub-arbore drept, el va fi element maxim într-un sub-arbore. Părintele rădăcinii acestui sub-arbore este nodul succesor.

Predecesorul în inordine

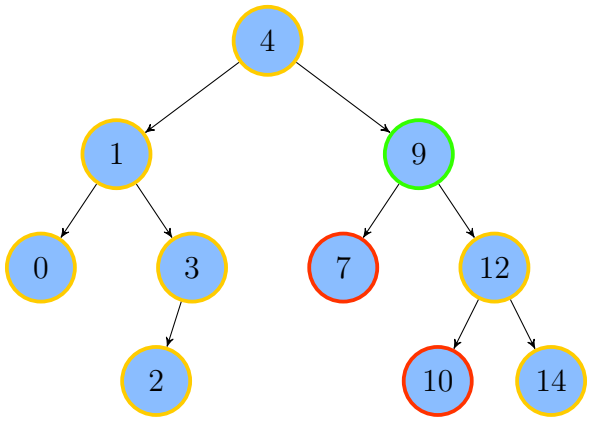
Similar, predecesorul unui nod, la traversarea în inordine, se definește ca:

1. elementul maxim din sub-arborele stâng al nodului, dacă nodul are sub-arbore stâng.
2. dacă nodul nu are sub-arbore stâng, el va fi element minim într-un sub-arbore. Părintele rădăcinii acestui sub-arbore este nodul predecesor.

Cerința 3 (2p) Implementația funcția `lowestCommonAncestor` care determină cel mai apropiat strămoș comun pentru două noduri având cheile `value1` și `value2`.

```
Tree lowestCommonAncestor(Tree root, Item value1, Item value2);
```

Pentru nodurile cu valorile 7 și 10, din arborele de mai jos, avem cel mai apropiat strămoș comun nodul cu valoarea 9.



3. Informații utile

Pentru toate cerințele, putem testa și individual funcțiile implementate. Pentru acest lucru, va trebui să rulăm executabilul `TestBST` utilizând argumente în linia de comandă:

- 1 – pentru a testa `insert` și `init` (1 punct);
- 2 – pentru a testa `isEmpty` (0.25 puncte);
- 3 – pentru a testa `contains` (0.25 puncte);
- 4 – pentru a testa `minimum` și `maximum` (1 punct);
- 5 – pentru a testa `successor` (1 punct);
- 6 – pentru a testa `predecessor` (1 punct);
- 7 – pentru a testa `delete` (2 puncte);
- 8 – pentru a testa `lca` (2 puncte);
- 9 – pentru a testa `destroyTree` - verifică dacă root devine NULL (0.75 puncte).

Observație: Punctajul suplimentar de 0.75 puncte pentru `destroyTree` (verificare memory leaks) se acordă automat la `make test` dacă nu există pierderi de memorie detectate cu `valgrind` (Linux) sau `leaks` (macOS).

Exemple de testări:

- Testarea funcției `insert`

```
$ ./TestBST 1
insert      - Toate testele au trecut!  Puncte: 1.00
SCORE:1.00
```

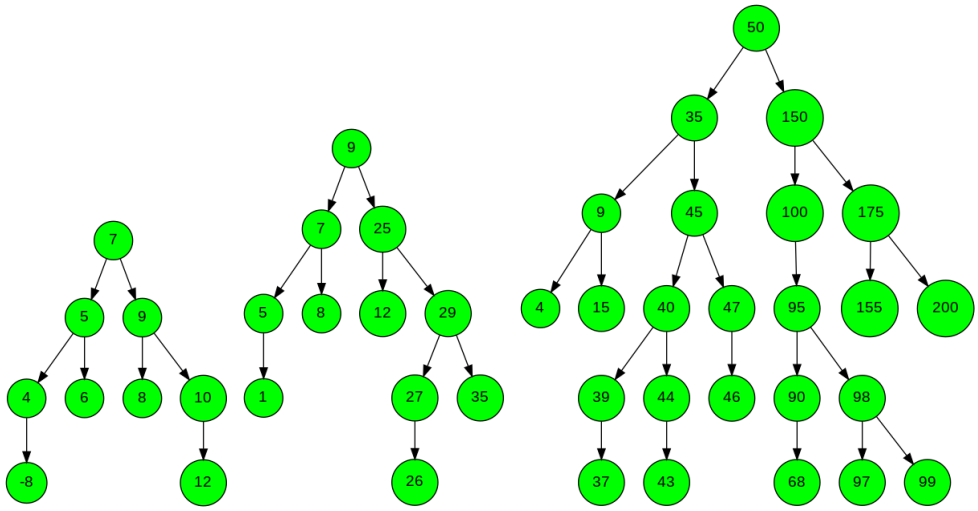
  Pentru a fi mai ușor de verificat dacă este corectă implementarea, este implementată o funcție care desenează arborele. Această funcție utilizează utilitarul `graphViz`.

  Pentru a putea instala acest utilitar pe Linux, putem folosi comanda:

```
sudo apt-get install graphviz
```

  Pentru a putea instala acest utilitar pe Windows, putem descărcă executabilul de la adresa: https://graphviz.org/download/.

  Exemple de imagini generate:



- Testarea tuturor funcțiilor folosind `make test`:

```
$ make test
insert      - Toate testele au trecut!  Puncte: 1.00
isEmpty     - Toate testele au trecut!  Puncte: 0.25
contains    - Toate testele au trecut!  Puncte: 0.25
minmax      - Toate testele au trecut!  Puncte: 1.00
successor   - Toate testele au trecut!  Puncte: 1.00
predecessor - Toate testele au trecut!  Puncte: 1.00
delete      - Toate testele au trecut!  Puncte: 2.00
lca         - Toate testele au trecut!  Puncte: 2.00
destroyTree - root == NULL? DA          Puncte: 0.75
========================================
Verificare Valgrind (destroyTree):
  PASSED - Fara memory leaks!
destroyTree                             Puncte: 0.75
========================================
Toate testele au trecut! Felicitari!
Punctajul obtinut este: 10.00
```